SmartTaskScheduler

1.Source code

Task.java

```
import java.io.Serializable;
import java.time.LocalDate;

public class Task implements Comparable<Task>, Serializable {

    private String title;
    private String description;
    private int priority; // 1 = High, 2 = Medium, 3 = Low
    private LocalDate deadline;
    private boolean completed;

    public Task(String title, String description, int priority, LocalDate deadline) {
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.deadline = deadline;
        this.completed = false;
    }

    public String getTitle() { return title; }
    public String getDescription() { return description; }
    public int getPriority() { return priority; }
    public LocalDate getDeadline() { return deadline; }
    public boolean isCompleted() { return completed; }

    public void markCompleted() {
        this.completed = true;
    }

    @Override
    public int compareTo(Task other) {
        if (this.priority != other.priority) {
            return this.priority - other.priority;
        }
        return this.deadline.compareTo(other.deadline);
    }
```

```
    @Override
    public String toString() {
        return title +
                " | Priority: " + priority +
                " | Deadline: " + deadline +
                " | Completed: " + completed;
    }
}
```

TaskManager.java

```
import java.io.*;
import java.time.LocalDate;
import java.util.*;

public class TaskManager {

    private PriorityQueue<Task> taskQueue;
    private final String FILE_NAME = "tasks.dat";

    public TaskManager() {
        taskQueue = new PriorityQueue<>();
        loadTasksFromFile();
    }

    public void addTask(Task task) {
        taskQueue.add(task);
        saveTasksToFile();
    }

    public void deleteTask(String title) {
        Iterator<Task> iterator = taskQueue.iterator();
        while (iterator.hasNext()) {
            Task = iterator.next();
            if (task.getTitle().equalsIgnoreCase(title)) {
                iterator.remove();
                saveTasksToFile();
                return;
            }
        }
```

```
}

public void markTaskCompleted(String title) {
    for (Task : taskQueue) {
        if (task.getTitle().equalsIgnoreCase(title)) {
            task.markCompleted();
            saveTasksToFile();
            return;
        }
    }
}

public List<Task> getAllTasks() {
    return new ArrayList<>(taskQueue);
}

public List<Task> getTodayTasks() {
    List<Task> todayTasks = new ArrayList<>();
    LocalDate today = LocalDate.now();
    for (Task : taskQueue) {
        if (task.getDeadline().equals(today)) {
            todayTasks.add(task);
        }
    }
    return todayTasks;
}

public List<Task> getHighPriorityTasks() {
    List<Task> highTasks = new ArrayList<>();
    for (Task : taskQueue) {
        if (task.getPriority() == 1) {
            highTasks.add(task);
        }
    }
    return highTasks;
}

// Reminder
public void startReminder() {
    Timer = new Timer();
    timer.schedule(new TimerTask() {
        @Override
```

```
            public void run() {
                Task next = taskQueue.peek();
                if (next != null && !next.isCompleted()) {
                    System.out.println("🔔 Reminder: Complete task -> " + next.getTitle());
                }
            }
        }, 5000, 10000);
    }

    private void saveTasksToFile() {
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(FILE_NAME))) {
            oos.writeObject(new ArrayList<>(taskQueue));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private void loadTasksFromFile() {
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(FILE_NAME))) {
            List<Task> list = (List<Task>) ois.readObject();
            taskQueue.addAll(list);
        } catch (Exception e) {
            System.out.println("No previous tasks found.");
        }
    }
}
```

TaskSchedulerUI.java

```
import javax.swing.*;
import javax.swing.event.ListSelectionEvent;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDate;
import java.util.List;

public class TaskSchedulerUI {

    private TaskManager manager = new TaskManager();
    private DefaultTableModel tableModel;
    private JTable table;
```

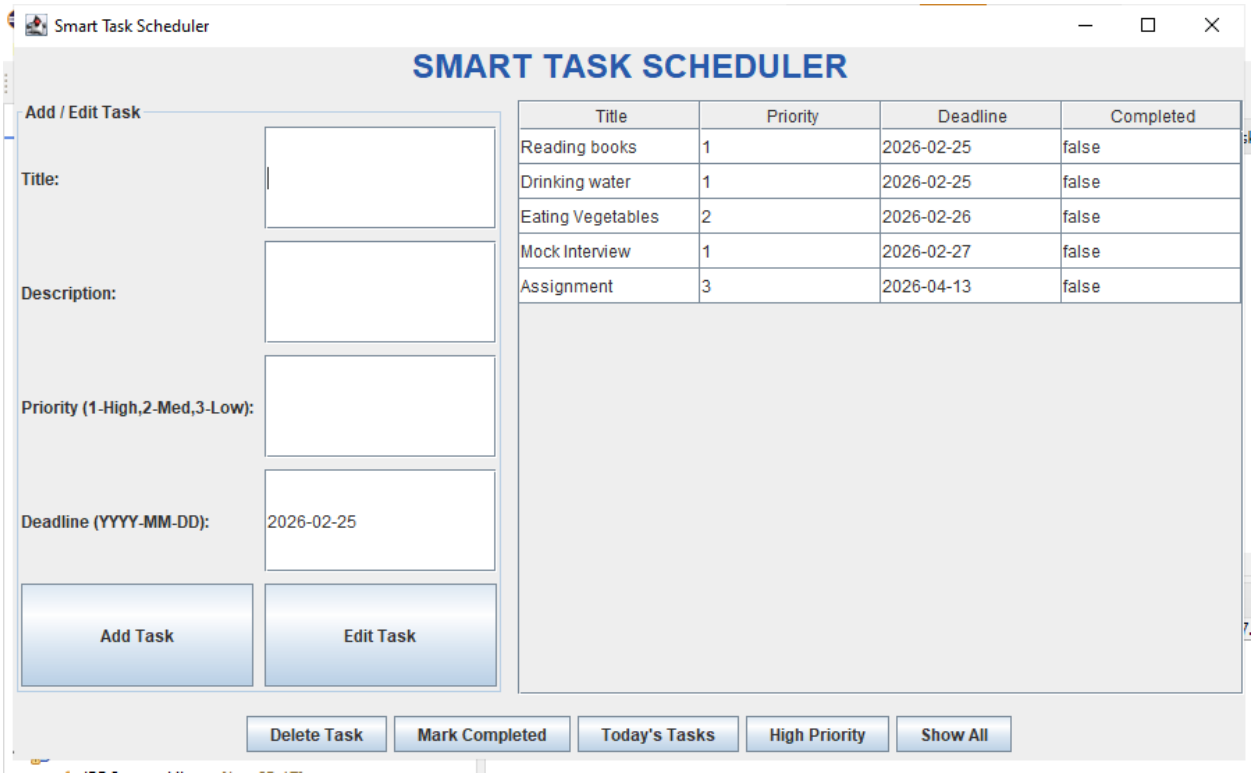
```
private JTextField titleField;
private JTextField descField;
private JTextField priorityField;
private JTextField deadlineField;

public TaskSchedulerUI() {

    JFrame frame = new JFrame("Smart Task Scheduler");
    frame.setSize(900, 550);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setLayout(new BorderLayout(10, 10));

    JLabel titleLabel = new JLabel("SMART TASK SCHEDULER", JLabel.CENTER);
    titleLabel.setFont(new Font("Arial", Font.BOLD, 24));
    titleLabel.setForeground(new Color(40, 90, 170));
    frame.add(titleLabel, BorderLayout.NORTH);

    String[] columns = {"Title", "Priority", "Deadline", "Completed"};
    tableModel = new DefaultTableModel(columns, 0);
    table = new JTable(tableModel);
    table.setRowHeight(25);
    frame.add(new JScrollPane(table), BorderLayout.CENTER);

    JPanel inputPanel = new JPanel(new GridLayout(5, 2, 8, 8));
    inputPanel.setBorder(BorderFactory.createTitledBorder("Add / Edit Task"));

    titleField = new JTextField();
    descField = new JTextField();
    priorityField = new JTextField();
    deadlineField = new JTextField("2026-02-25");

    inputPanel.add(new JLabel("Title:"));
    inputPanel.add(titleField);
    inputPanel.add(new JLabel("Description:"));
    inputPanel.add(descField);
    inputPanel.add(new JLabel("Priority (1-High,2-Med,3-Low):"));
    inputPanel.add(priorityField);
    inputPanel.add(new JLabel("Deadline (YYYY-MM-DD):"));
    inputPanel.add(deadlineField);

    JButton addButton = new JButton("Add Task");
```

```
JButton editButton = new JButton("Edit Task");
JButton deleteButton = new JButton("Delete Task");
JButton completeButton = new JButton("Mark Completed");

inputPanel.add(addButton);
inputPanel.add(editButton);

frame.add(inputPanel, BorderLayout.WEST);

JPanel bottomPanel = new JPanel();
JButton todayButton = new JButton("Today's Tasks");
JButton highButton = new JButton("High Priority");
JButton showAllButton = new JButton("Show All");

bottomPanel.add(deleteButton);
bottomPanel.add(completeButton);
bottomPanel.add(todayButton);
bottomPanel.add(highButton);
bottomPanel.add(showAllButton);

frame.add(bottomPanel, BorderLayout.SOUTH);

loadTable(manager.getAllTasks());

table.getSelectionModel().addListSelectionListener((ListSelectionEvent event) -> {
   int selectedRow = table.getSelectedRow();
   if (selectedRow != -1) {
      titleField.setText(tableModel.getValueAt(selectedRow, 0).toString());
      priorityField.setText(tableModel.getValueAt(selectedRow, 1).toString());
      deadlineField.setText(tableModel.getValueAt(selectedRow, 2).toString());
   }
});

addButton.addActionListener(e -> {
   try {
      Task = new Task(
            titleField.getText(),
            descField.getText(),
            Integer.parseInt(priorityField.getText()),
            LocalDate.parse(deadlineField.getText())
      );
      manager.addTask(task);
```

```
        loadTable(manager.getAllTasks());
        clearFields();
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(frame, "Invalid Input!");
    }
});

editButton.addActionListener(e -> {
    int selectedRow = table.getSelectedRow();
    if (selectedRow != -1) {
        try {
            String oldTitle = tableModel.getValueAt(selectedRow, 0).toString();
            manager.deleteTask(oldTitle);

            Task updatedTask = new Task(
                    titleField.getText(),
                    descField.getText(),
                    Integer.parseInt(priorityField.getText()),
                    LocalDate.parse(deadlineField.getText())
            );

            manager.addTask(updatedTask);
            loadTable(manager.getAllTasks());
            clearFields();
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(frame, "Invalid Input!");
        }
    }
});

deleteButton.addActionListener(e -> {
    int selectedRow = table.getSelectedRow();
    if (selectedRow != -1) {
        String title = tableModel.getValueAt(selectedRow, 0).toString();
        manager.deleteTask(title);
        loadTable(manager.getAllTasks());
        clearFields();
    }
});

completeButton.addActionListener(e -> {
    int selectedRow = table.getSelectedRow();
```

```
            if (selectedRow != -1) {
                String title = tableModel.getValueAt(selectedRow, 0).toString();
                manager.markTaskCompleted(title);
                loadTable(manager.getAllTasks());
                clearFields();
            }
        });

        todayButton.addActionListener(e -> loadTable(manager.getTodayTasks()));
        highButton.addActionListener(e -> loadTable(manager.getHighPriorityTasks()));
        showAllButton.addActionListener(e -> loadTable(manager.getAllTasks()));

        manager.startReminder();
        frame.setVisible(true);
    }

    private void loadTable(List<Task> tasks) {
        tableModel.setRowCount(0);
        for (Task : tasks) {
            tableModel.addRow(new Object[]{
                    task.getTitle(),
                    task.getPriority(),
                    task.getDeadline(),
                    task.isCompleted()
            });
        }
    }

    private void clearFields() {
        titleField.setText("");
        descField.setText("");
        priorityField.setText("");
        deadlineField.setText("");
    }

    public static void main(String[] args) {
        new TaskSchedulerUI();
    }
}
```

2.Output:


Smart Task Scheduler
SMART TASK SCHEDULER
Add / Edit Task
Title:
Description:
Priority (1-High,2-Med,3-Low):
Deadline (YYYY-MM-DD):
2026-02-25
Add Task
Edit Task
Title
Priority
Deadline
Completed
Reading books
1
2026-02-25
false
Drinking water
1
2026-02-25
false
Eating Vegetables
2
2026-02-26
false
Mock Interview
1
2026-02-27
false
Assignment
3
2026-04-13
false
Delete Task
Mark Completed
Today's Tasks
High Priority
Show All